Databases and its Applications

Relational Modelling

Pooja T S

Computer Applications

Databases and its Applications

Relational Modelling

SQL: Views and Indexing Techniques

Pooja T S

Computer Applications

Databases and its Applications

Introduction to Views and Indexing

PES UNIVERSITY

- In real databases, data is often reused, aggregated, or secured for limited access.
- PostgreSQL provides two powerful features:
  - **Views** – virtual tables that store reusable SQL queries.
  - **Indexes** – data structures that speed up query performance.
- Both play a crucial role in database optimisation and abstraction.

What is a View?

- A **View** is a saved SQL query represented as a virtual table.
- It stores no data itself; instead, it fetches data dynamically from base tables.
- Acts as a security layer and simplifies complex queries.

Databases and its Applications

Purpose of Using Views

PES UNIVERSITY

- Common use-cases include:
  - Hiding sensitive columns from users.
  - Simplifying repetitive complex joins or aggregations.
  - Providing customised representations of data.
  - Enforcing read-only or restricted data access.

CREATE VIEW – Syntax

- Syntax: `CREATE VIEW view_name AS SELECT columns FROM table_name WHERE condition;`
- Use `DROP VIEW view_name;` to remove a view.
- Use `CREATE OR REPLACE VIEW` to modify an existing one.

CREATE VIEW – Example

- **Command:**

```
CREATE VIEW high_achievers AS
SELECT sname, deptid, cgpa
FROM student
WHERE cgpa > 8.5;
```

- **Output:**

```
CREATE VIEW
```

- The view now behaves like a table for future queries.

Querying a View

- **Command:**

```
SELECT * FROM high_achievers;
```

- **Output (sample):**

```
 sname | deptid | cgpa
-------+--------+------
 Kavya | 2 | 9.10
(1 row)
```

- The query executes dynamically against underlying tables.

Databases and its Applications

Modifying Data through Views

- Some simple views in PostgreSQL are **updatable**.
- Data modifications on the view affect the base table directly.
- Example: `UPDATE high_achievers SET cgpa = 9.2 WHERE sname = 'Kavya';`
- Use caution — not all complex joins or aggregates are updatable.

Databases and its Applications

Materialized Views

- A **Materialized View** stores the query result physically on disk.
- Useful for slow-running aggregation queries.
- Syntax: `CREATE MATERIALIZED VIEW view_name AS SELECT ...;`
- To refresh: `REFRESH MATERIALIZED VIEW view_name;`
- Data remains static until explicitly refreshed.

Databases and its Applications

Advantages and Limitations of Views

PES UNIVERSITY

- **Advantages:**
  - Enhances security and simplifies access.
  - Reuses query logic without duplication.
  - Improves readability for complex joins.
- **Limitations:**
  - Cannot index a standard (virtual) view directly.
  - Complex nested views can slow down queries.
  - Must refresh materialized views to stay up-to-date.

Databases and its Applications

What is an Index?

- An **Index** is a data structure that improves the speed of data retrieval on a table.
- It functions like an index in a book — allows PostgreSQL to locate rows faster.
- Common index types: **B-tree**, **Hash**, **GIN**, **GiST**.

Databases and its Applications

Need for Indexes

- Without indexes, PostgreSQL performs sequential scans.
- As tables grow large, sequential scans become inefficient.
- Indexes improve performance of:
  - `WHERE` clause searches.
  - `JOIN` and `ORDER BY` operations.
  - `DISTINCT` and `GROUP BY` queries.

CREATE INDEX – Syntax

- Syntax: `CREATE INDEX index_name ON table_name (column_name);`
- Example: `CREATE INDEX idx_student_deptid ON student(deptid);`
- Output: `CREATE INDEX`

Databases and its Applications

Checking Existing Indexes

- Use PostgreSQL meta-commands:
  - `\di` – list all indexes.
  - `\d table_name` – shows table indexes.
- Output (sample):

```
List of relations
Schema | Name | Type | Owner
--------+------+-------+--------
public | idx_student_deptid | index | postgres
```

Understanding EXPLAIN and ANALYZE in PostgreSQL

- PostgreSQL provides tools to analyse how queries are executed internally.
- **EXPLAIN** displays the execution plan chosen by the query planner.
- **ANALYZE** runs the query and adds actual execution time and row counts.
- Together, they help evaluate the performance and index usage of queries.
- Syntax:
  - `EXPLAIN SELECT * FROM student WHERE deptid = 3;`
  - `EXPLAIN ANALYZE SELECT * FROM student WHERE deptid = 3;`

Databases and its Applications

How PostgreSQL Execution Plans Work

PES UNIVERSITY

- PostgreSQL's query planner estimates:
  - Cost of scanning each table.
  - Number of rows expected.
  - Whether an index is beneficial.
- Output example (simplified):
  - `Seq Scan on student (cost=0.00..12.50 rows=3 width=48)`
  - `Index Scan using idx_student_deptid (cost=0.14..8.27 rows=2 width=48)`
- **Interpretation:**
  - "Seq Scan" → sequential scan (no index used).
  - "Index Scan" → uses the created index for faster access.

Databases and its Applications

Effect of Index on Query Performance

- Before indexing: `EXPLAIN ANALYZE SELECT * FROM student WHERE deptid = 3;`
- Output: sequential scan on table (slow).
- After creating index: `EXPLAIN ANALYZE SELECT * FROM student WHERE deptid = 3;`
- Output: index scan on `idx_student_deptid` (faster).
- PostgreSQL automatically uses the most efficient plan.

Databases and its Applications

DROP and REINDEX Commands

PES UNIVERSITY

- **Drop an index:**

```
DROP INDEX idx_student_deptid;
```

- **Rebuild an index:**

```
REINDEX TABLE student;
```

- **Notes:**
  - Dropping an index does not affect table data.
  - Reindexing is used for maintenance or corruption recovery.

Databases and its Applications

Composite and Unique Indexes

- PostgreSQL supports multi-column and uniqueness-enforcing indexes.
- Examples:
  - `CREATE UNIQUE INDEX idx_student_sname_deptid ON student(sname, deptid);`
  - Prevents duplicate student names in the same department.

Databases and its Applications

Partial and Expression Indexes

- **Partial index:** `CREATE INDEX idx_highcgpa ON student(cgpa) WHERE cgpa > 8.0;`
- Indexes only qualifying rows — saves space and time.
- **Expression index:** `CREATE INDEX idx_lower_name ON student((lower(sname)));`
- Enables fast case-insensitive searches.

Databases and its Applications

Advantages and Maintenance of Indexes

- **Advantages:**
  - Faster searches and joins.
  - Improved sorting and grouping.
- **Maintenance:**
  - Indexes consume extra disk space.
  - They slow down `INSERT` and `UPDATE` slightly.
  - Use `VACUUM` and `REINDEX` periodically.

Databases and its Applications

Best Practices – Views and Indexes

- For **Views:**
  - Keep definitions simple for performance.
  - Use materialized views for heavy aggregations.
  - Avoid chaining too many nested views.
- For **Indexes:**
  - Index columns frequently used in `WHERE`, `JOIN`, or `ORDER BY`.
  - Avoid indexing small tables.
  - Test query plans with `EXPLAIN ANALYZE`.

Databases and its Applications

Summary – Views and Indexing

PES UNIVERSITY

- You have learned:
  - What views and indexes are in PostgreSQL.
  - Syntax and examples for `CREATE`, `DROP`, and `REINDEX`.
  - Difference between normal and materialized views.
  - Index types: simple, unique, composite, partial, and expression-based.
- Together, these features improve database security, performance, and scalability.

Databases and its Applications

Extending SQL – DCL and TCL Overview

- Beyond DDL and DML, two other important SQL command groups are:
  - **DCL – Data Control Language** Controls access and permissions on database objects.
  - **TCL – Transaction Control Language** Manages changes made by DML statements within transactions.
- Both are supported in PostgreSQL for multi-user and secure database operations.

Databases and its Applications

DCL – Data Control Language

PES UNIVERSITY

- DCL commands control who can access or modify database objects.
- Common DCL commands:
  - `GRANT` – gives privileges to users.
  - `REVOKE` – removes privileges.
- Example: `GRANT SELECT, INSERT ON student TO faculty_user;`
- Output: `GRANT`
- Revoking permissions: `REVOKE INSERT ON student FROM faculty_user;`

Databases and its Applications

Understanding PostgreSQL Roles and Privileges

PES UNIVERSITY

- PostgreSQL manages users through **roles**.
- A role can own tables, run queries, or be granted limited access.
- **Typical privileges:**
  - `SELECT`, `INSERT`, `UPDATE`, `DELETE`
  - `ALL PRIVILEGES` – full control over the object
- To view privileges: `\z student`

Databases and its Applications

TCL – Transaction Control Language

PES UNIVERSITY

- TCL commands manage groups of DML operations as a single logical unit.
- They ensure **Atomicity**, **Consistency**, **Isolation**, and **Durability** (ACID).
- Common TCL commands:
  - `BEGIN` – starts a transaction block.
  - `COMMIT` – saves all changes.
  - `ROLLBACK` – undoes all uncommitted changes.
  - `SAVEPOINT` – sets a checkpoint within a transaction.

TCL Example – COMMIT and ROLLBACK

- Example in PostgreSQL:
  - `BEGIN;`
  - `UPDATE student SET cgpa = cgpa + 0.3 WHERE deptid = 2;`
  - `ROLLBACK;`
- **Output:** `ROLLBACK` (All changes undone)
- If instead you use `COMMIT;` → changes are made permanent.

Databases and its Applications

TCL Example – SAVEPOINT Usage

- Example:
  - `BEGIN;`
  - `UPDATE student SET cgpa = cgpa + 0.2 WHERE deptid = 1;`
  - `SAVEPOINT sp1;`
  - `DELETE FROM student WHERE deptid = 3;`
  - `ROLLBACK TO sp1;`
  - `COMMIT;`
- Effect:
  - Changes before `SAVEPOINT sp1` are preserved.
  - Operations after it are undone.

Databases and its Applications

Summary – DCL and TCL

- **DCL:**
  - Grants and revokes user permissions.
  - Controls access to database objects.
- **TCL:**
  - Ensures reliability of multiple DML operations.
  - Supports rollback and partial transaction management.
- Together, they ensure secure, consistent, and controlled database operations.

Thank You

Pooja T S

Assistant Professor

Department of Computer Applications

poojats@pes.edu

080-26721983 Extn: 233